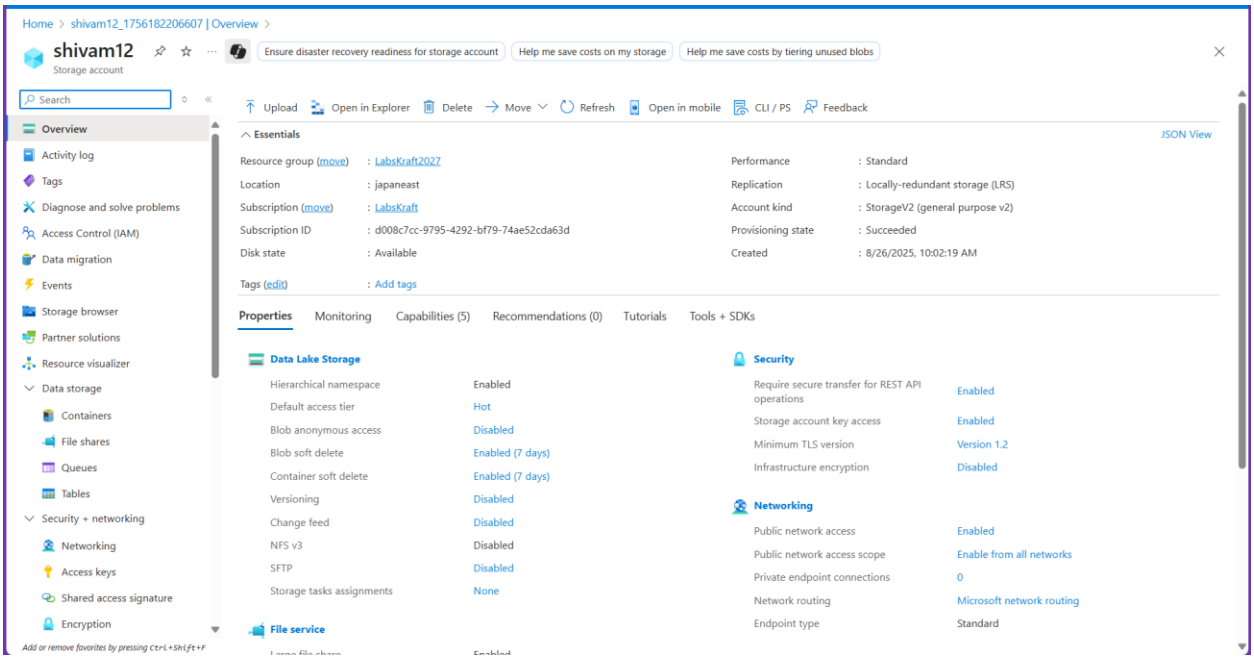
# Project 02: Retail Compliance & Analytics Platform on Azure Databricks

**Shivam Adbhute**

**HERE IS THE STEP BY STEP APPROACH THAT I FOLLOWED -**

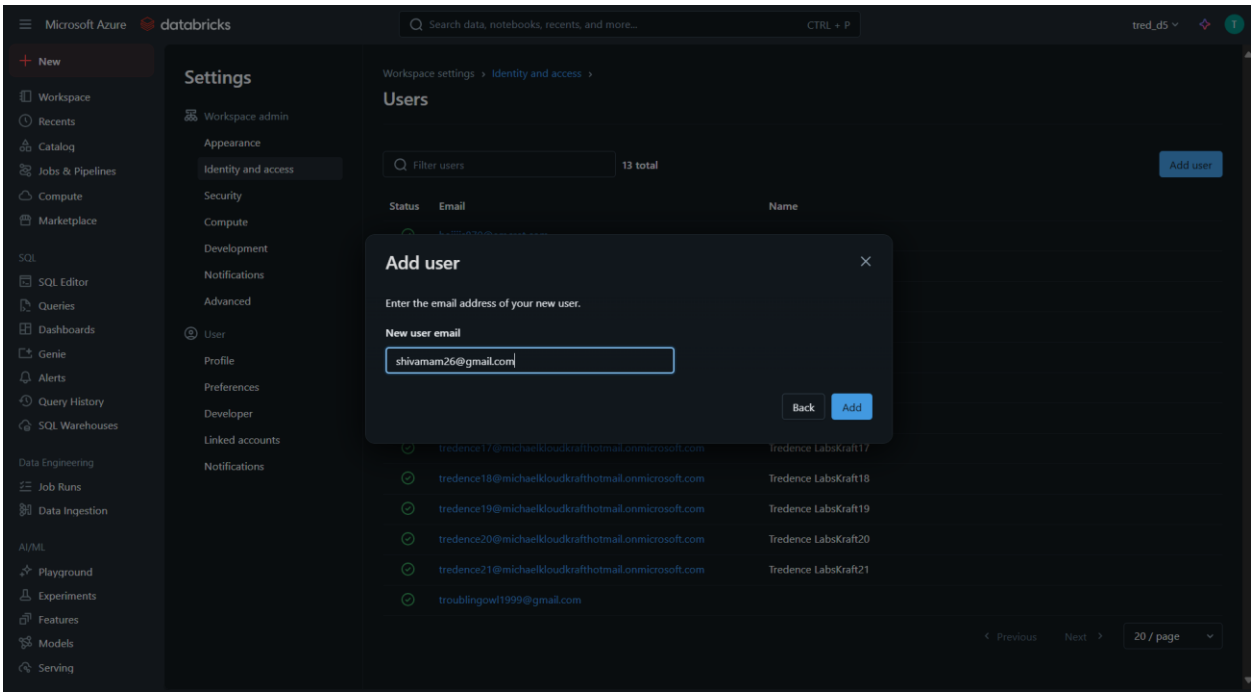
### Step 1: Create Azure Storage Account and Workspace

- Set up a new Azure Storage account.
- Create a Databricks workspace linked to the storage account



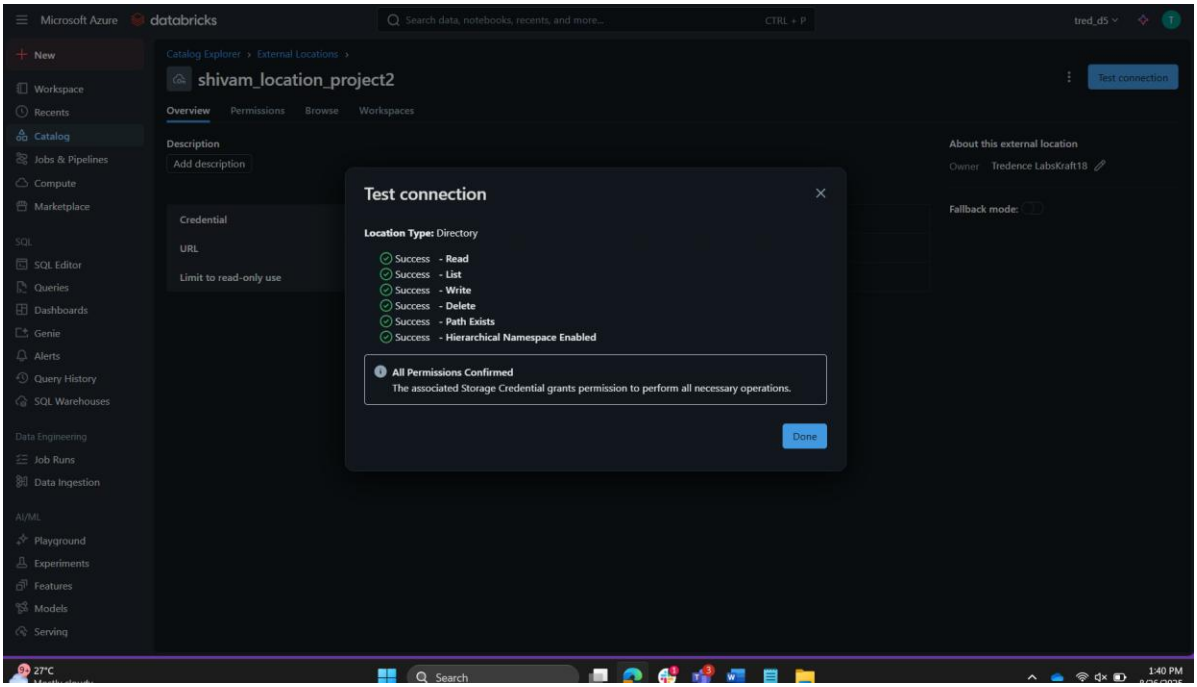
**Step 2: Add Users to Workspace**

- Assign users and groups to the workspace using Role-Based Access Control (RBAC).
- Ensure appropriate permissions are granted for collaboration.



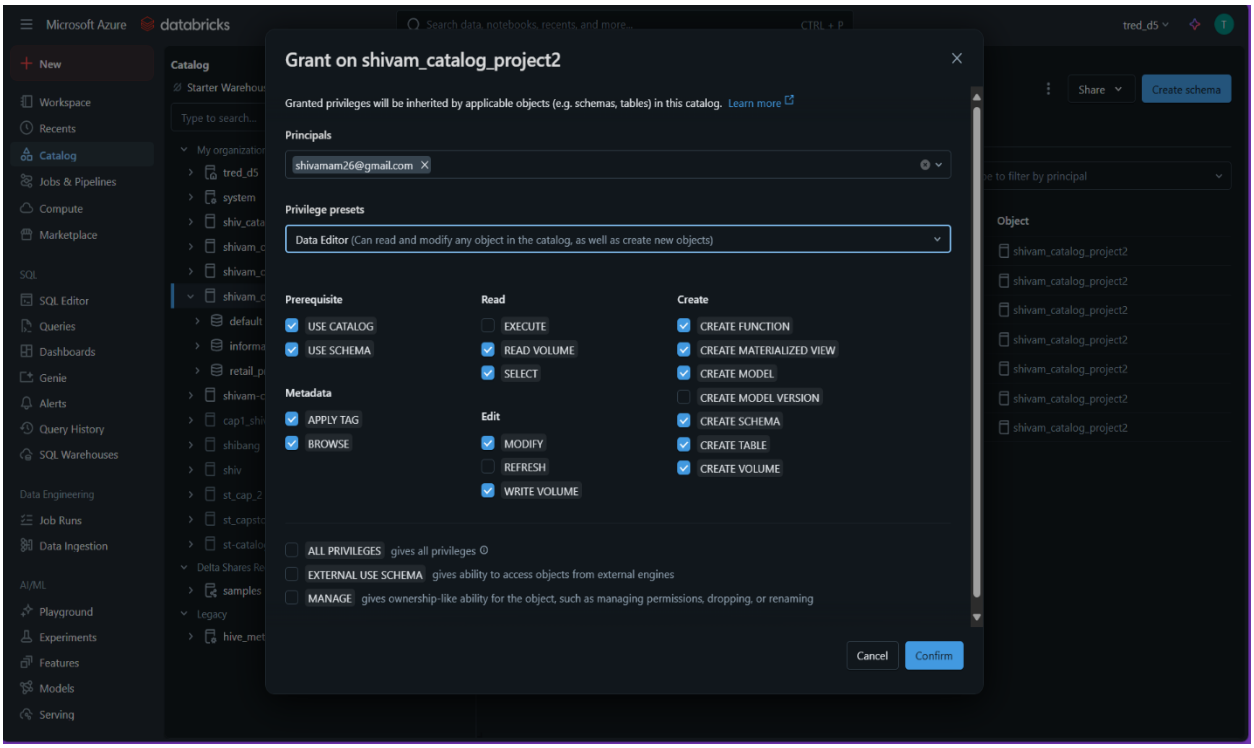

**Step 3: Configure External Location**

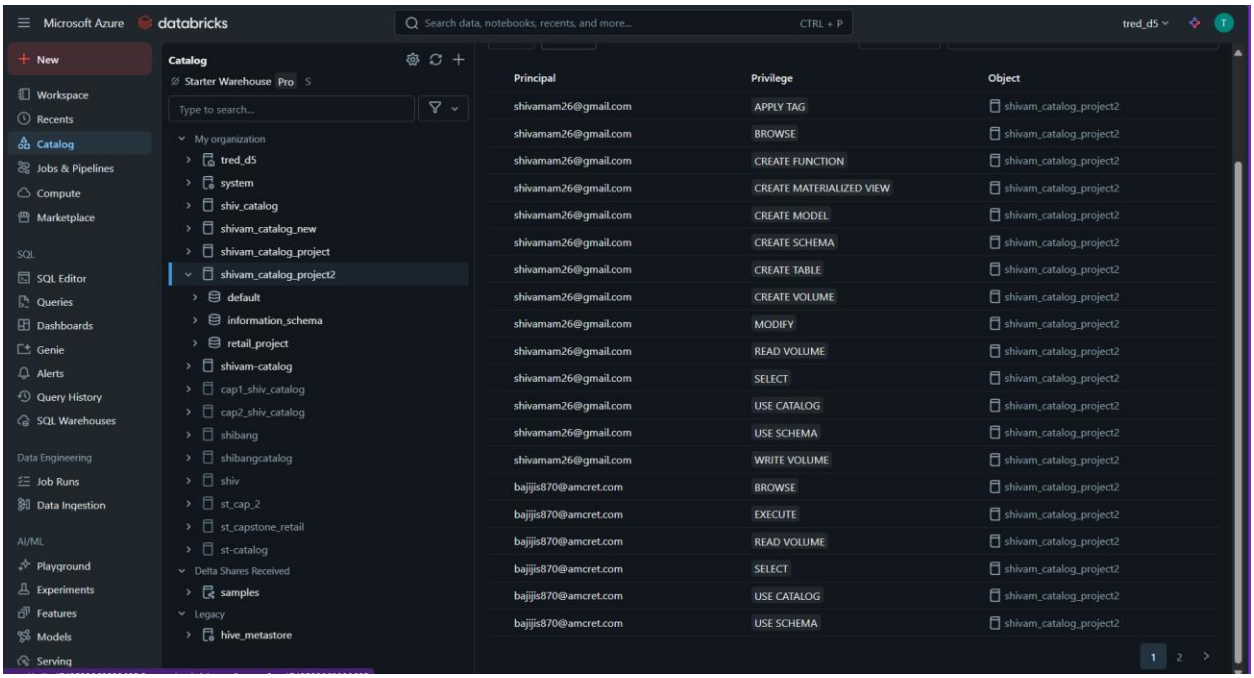
- Define external locations for raw data ingestion.
- Connect to storage containers for sales and product data.




### Step 4: Provision Users and Groups

- Use RBAC to manage access to resources.
- Assign roles such as Contributor, Reader, and Owner as needed.






**Step 5: Build DLT Pipeline**

**Create a Delta Live Table (DLT) pipeline named retail_sales_pipeline.**

**Structure the pipeline into Bronze, Silver, and Gold layers.**

**CODE FOR INGESTION –**

```python
import dlt
from pyspark.sql.functions import *
from pyspark.sql.types import to_date

# Ingest raw sales data (multiple daily CSV files in raw/sales/)
@dlt.table(
  name="sales_raw",
  comment="Raw sales data ingested from CSV files",
  table_properties={"quality": "bronze"}
)
def sales_raw():
    return (
        spark.readStream.format("cloudFiles")     # Auto Loader for incremental loads
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://shivam12@shivamraw.dfs.core.windows.net/sales/")
    )

# Products CSV
@dlt.table(
  name="products_raw_csv",
  comment="Raw product data from CSV",
  table_properties={"quality": "bronze"}
)
def products_raw_csv():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "csv")
        .option("header", "true")
        .load("abfss://shivam12@shivamraw.dfs.core.windows.net/products/")
    )

# Products JSON
@dlt.table(
  name="products_raw_json",
  comment="Raw product data from JSON",
  table_properties={"quality": "bronze"}
)
def products_raw_json():
    return (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", "json")
        .load("abfss://shivam12@shivamraw.dfs.core.windows.net/products/")
    )

# Unified Products (CSV + JSON)
@dlt.view(name="products_raw")
def products_raw():
    return dlt.read("products_raw_csv").unionByName(dlt.read("products_raw_json"),
allowMissingColumns=True)

# Clean Sales
@dlt.table(
  name="sales_clean",
  comment="Cleaned sales data with enforced schema",
  table_properties={"quality": "silver"}
)
@dlt.expect("valid_sales_amount", "sales_amount >= 0")
@dlt.expect("valid_quantity", "quantity > 0")
def sales_clean():
    return (
        dlt.read("sales_raw")
        .withColumn("sale_id", col("sale_id").cast("string"))
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("region", col("region").cast("string"))
```

```
        .withColumn("store_id", col("store_id").cast("string"))
        .withColumn("quantity", col("quantity").cast("int"))
        .withColumn("sales_amount", col("sales_amount").cast("double"))
        .withColumn("sale_date", to_date(col("sale_date"), "yyyy-MM-dd"))
        .na.drop()
    )

@dlt.table(
  name="products_clean",
  comment="Cleaned products data with schema evolution support",
  table_properties={"quality": "silver"}
)
def products_clean():
    return (
        dlt.read("products_raw")
        .withColumn("product_id", col("product_id").cast("string"))
        .withColumn("product_name", col("product_name").cast("string"))
        .withColumn("category", col("category").cast("string"))
        .withColumn("price", col("price").cast("double"))
        .withColumn("discount_rate", col("discount_rate").cast("double"))
    )

# Join sales + products and calculate KPIs
@dlt.table(
  name="sales_summary",
  comment="Daily sales summary by product and region",
  table_properties={"quality": "gold"}
)
def sales_summary():
    sales = dlt.read("sales_clean")
    products = dlt.read("products_clean")
    return (
        sales.join(products, "product_id", "left")
        .groupBy("region", "sale_date", "product_id", "product_name", "category")
        .agg(
            sum("sales_amount").alias("daily_revenue"),
            sum("quantity").alias("units_sold"),
            avg("price").alias("avg_price"),
            avg("discount_rate").alias("avg_discount")
        )
    )
```

**Step 6: Ingest Raw Data (Bronze Layer)**

- **Use Auto Loader to ingest CSV and JSON files from external storage.**
- **Define DLT tables:**
    - **sales_raw**
    - **products_raw_csv**
    - **products_raw_json**

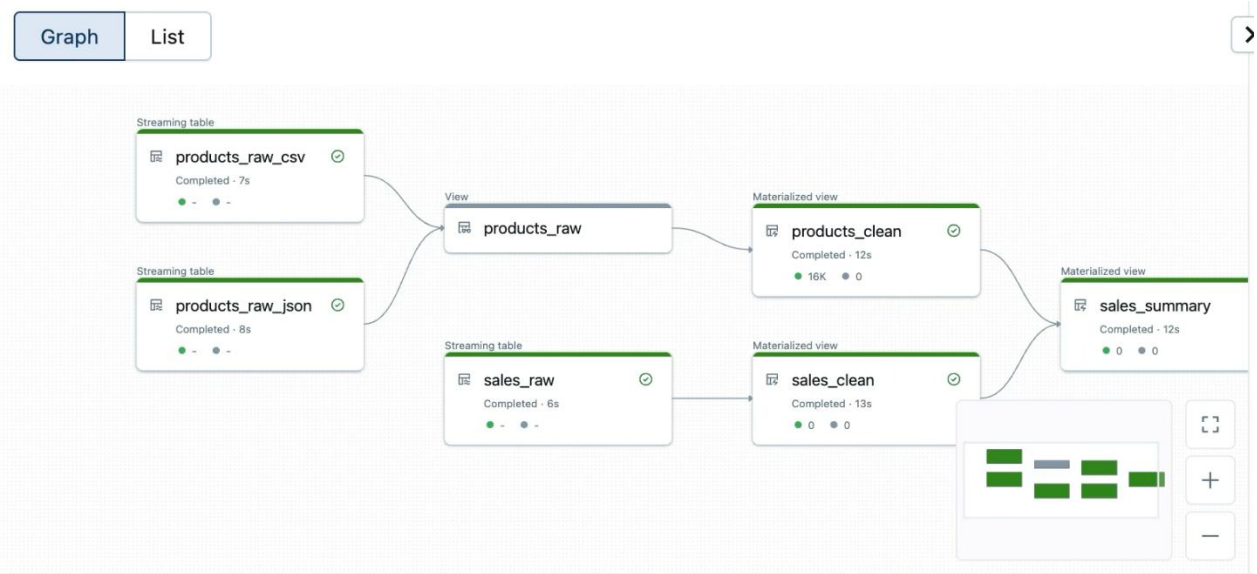
- **products_raw (Unified view)**

**Step 7: Clean and Transform Data (Silver Layer)**

- **Apply schema enforcement and data validation:**
  - **sales_clean with expectations on sales_amount and quantity.**
  - **products_clean with schema evolution support.**

**Step 8: Aggregate and Analyze Data (Gold Layer)**

- **Join sales and product data.**
- **Create sales_summary table with KPIs:**
  - **Daily revenue**
  - **Units sold**
  - **Average price**
  - **Average discount**




Currently we are not use **schema evolution so if we make any changes to schema we will encounter an error**


**Error details**

com.databricks.pipelines.common.errors.DLTSparkException: [FLOW_SCHEMA_CHANGED] Flow pratham_cat_2608.pratham_schema.sales_raw has terminated since it encountered a schema change during execution.
The schema change is compatible with existing target schema and the next run of the flow can resume with the new schema.

org.apache.spark.sql.streaming.StreamingQueryException: [STREAM_FAILED] Query [id = 48d72fe9-52bf-4600-9743-30a1f3c9eef7, runId = 0f568aac-0455-4c32-83b5-e377c039ae7a] terminated with exception: [UNKNOWN_FIELD_EXCEPTION.NEW_FIELDS_IN_FILE] Encountered unknown fields during parsing: [sales_amt], which can be fixed by an automatic retry: true
Unknown fields inside file path: abfss://raw@prathamstorage1.dfs.core.windows.net/sales/sales_day_6.csv
Rescued file schema: sales_amt STRING SQLSTATE: KD003 SQLSTATE: XXKST
=== Streaming Query ===
Identifier: pratham_cat_2608.pratham_schema.__materialization_mat_d401be79_fcf1_4d72_b799_c7a0da54dce8_sales_raw_1 [id = 48d72fe9-52bf-4600-9743-30a1f3c9eef7, runId = 0f568aac-0455-4c32-83b5-e377c039ae7a]
Current Start Offsets: {CloudFilesSource[abfss://raw@prathamstorage1.dfs.core.windows.net/sales/]: {"seqNum":7,"sourceVersion":3,"lastBackfillStartTimeMs":1756186897484,"lastBackfillFinishTimeMs":1756186901366,"lastInputPath":"abfss://raw@prathamstorage1.dfs.core.windows.net/sales/"}}
Current End Offsets: {CloudFilesSource[abfss://raw@prathamstorage1.dfs.core.windows.net/sales/]: {"seqNum":9,"sourceVersion":3,"lastBackfillStartTimeMs":1756186897484,"lastBackfillFinishTimeMs":1756186901366,"lastInputPath":"abfss://raw@prathamstorage1.dfs.core.windows.net/sales/"}}

**Now we will update the code**

.option("cloudFiles.inferColumnTypes", "true")  # infer schema automatically

.option("cloudFiles.schemaEvolutionMode", "addNewColumns")

SQL queries to evaluate –

-- Check if new rows landed in silver (clean data)

```
SELECT *
FROM shivam_catalog_project2.shivam_schema.sales_clean
WHERE sale_date = '2025-08-26';

-- Check Gold aggregation
SELECT *
FROM shivam_catalog_project2.shivam_schema.sales_summary
WHERE sale_date = '2025-08-26';

-- Check event log again (should now show success)
SELECT timestamp, level, message
FROM shivam_catalog_project2.shivam_schema.event_log_pratham
ORDER BY timestamp DESC
LIMIT 20;
```

Error log -

| timestamp | level | message |
|---|---|---|
| 2025-08-26T07:08:29.046+00:00 | METRICS | Reported cluster resources metrics. |
| 2025-08-26T07:07:29.046+00:00 | METRICS | Reported cluster resources metrics. |
| 2025-08-26T07:06:29.046+00:00 | METRICS | Reported cluster resources metrics. |
| 2025-08-26T07:05:29.046+00:00 | METRICS | Reported cluster resources metrics. |
| 2025-08-26T07:04:29.046+00:00 | METRICS | Reported cluster resources metrics. |
| 2025-08-26T07:03:35.886+00:00 | INFO | Update 04538a is COMPLETED. |
| 2025-08-26T07:03:35.389+00:00 | METRICS | Reported flow time metrics for flowName: 'pipelines.flowTimeMetrics.missingFlowName'. |
| 2025-08-26T07:03:35.388+00:00 | METRICS | Reported flow time metrics for flowName: 'shivam_catalog_project2.shivam_schema.products_raw_csv'. |
| 2025-08-26T07:03:35.386+00:00 | METRICS | Reported flow time metrics for flowName: 'shivam_catalog_project2.shivam_schema.sales_clean'. |

| timestamp | level | message |
|---|---|---|
| 2025-08-26T07:03:35.385+00:00 | METRICS | Reported flow time metrics for flowName: 'shivam_catalog_project2.shivam_schema.products_raw_json'. |
| 2025-08-26T07:03:35.384+00:00 | METRICS | Reported flow time metrics for flowName: 'shivam_catalog_project2.shivam_schema.sales_raw'. |
| 2025-08-26T07:03:35.383+00:00 | METRICS | Reported flow time metrics for flowName: 'shivam_catalog_project2.shivam_schema.sales_summary'. |

## Step 12: Visualize with Power BI

- Connect Power BI to the Gold layer.
- Create dashboards for sales performance and product insights.